Table of Contents

Serialization: What Each Namespace Does, and What It Costs When It Is Lost (2026-11-01)
- TL;DR
- 1. Method
  - 1.1 What this report measures
  - 1.2 Two kinds of evidence, and why both are needed
  - 1.3 The corpus
- 2. The namespace inventory
  - 2.1 What each namespace is
  - 2.2 Four traps built into the corpus
- 3. Where each editor's serialization boundary is
  - 3.1 SuperDoc — a proprietary engine with an open contract layer
  - 3.2 Folio — a total dispatch table over declared children, with a verbatim sink
  - 3.3 GenOffice — a byte-range splice that never re-serializes
  - 3.4 eigenpal — a typed vocabulary plus a generic catch-all
  - 3.5 office-open — a prefix-literal switch with a silent default
  - 3.6 Docen — a disposition table over office-open's model
  - 3.7 WordCraft — a typed Rust story model that regenerates what it models
  - 3.8 Plate.js — a closed passive vocabulary
  - 3.9 Fidus Writer — a literal-prefix if/else chain
  - 3.10 canvas-editor + canvas-editor-plugin — a prefix-literal DOM dispatch with no else
  - 3.11 docs.plus — Mammoth again, but the reporting layer is better
  - 3.12 Docxodus — an OpenXML SDK that draws its boundary around parts, not around vocabularies
- 4. Measured namespace survival
  - 4.1 How the measurement works
  - 4.2 The survival matrix
  - 4.3 Partial losses, name by name
  - 4.4 Where the static reading and the measurement disagree
  - 4.5 The code reading is wrong in both directions
- 5. The consequences, namespace by namespace
  - 5.1 mc — Markup Compatibility: which branch to believe
  - 5.2 w16du — Word 2023 UTC dates: one attribute, four uses
  - 5.3 w14 — the identity namespace
  - 5.4 w15, w16cid, w16cex — the three smallest extensions
  - 5.5 dgm — SmartArt: the four-part diagram

5.6 c — the chart, and its two orphaned companions
5.7 v and o — the legacy namespaces
5.8 m — OMML math
5.9 ns0 — the foreign namespace
5.10 w and r — the main vocabulary and the pointers
5.11 app, cp, dc, dcterms — the metadata
6. The cross-editor matrix
7. What this means
Reproducing this
Gaps and honest limits

Serialization: What Each Namespace Does, and What It Costs When It Is Lost (2026-11-01)

A companion to *Open-Source DOCX Editors, Re-verified Against Code and Measured by Round-Trip (2026-11-01)*, and the first half of what used to be *Report A*. That report asked how faithfully each editor preserved a document, and answered with a single number. This one opens the question up: **a DOCX file is not one thing but twenty-seven vocabularies, and an editor does not preserve or lose "a document" — it preserves or loses each vocabulary, in its own way, for its own reason.**

This report goes through the corpus namespace by namespace. For each one it states what the namespace *is* in WordprocessingML, what each of the **twelve editors** does with it **according to its own source code**, and what each editor **actually did** to it, measured by round-tripping the corpus through that editor and diffing the resulting package. A **thirteenth participant, Docxodus**, is measured the same way; it is not an editor but an OpenXML SDK, and §3.12 explains why its boundary is a different *kind* of boundary. The two kinds of evidence are reported separately because they are not the same claim, and in several places they disagree — in **both** directions, which is the part that was not obvious until the matrix was rebuilt against the measurement (§4.5).

TL;DR

- **Twenty-seven namespace URIs, 309 distinct `w:` element and attribute names, and 35 package parts.** The main WordprocessingML vocabulary is the `w:` names an editor's story has to carry; the other 26 URIs are what make a document a *document* rather than a wall of paragraphs.
- **The boundaries are not where the documentation says.** Read the source of all twelve editors and you find **five distinct kinds of serialization boundary**: a *typed vocabulary plus a generic catch-all* (eigenpal), a *total dispatch table over declared children with a verbatim sink for everything else* (Folio), a *byte-range splice* that never re-serializes at all (GenOffice), a *prefix-literal if/else chain* (Fidus, canvas-editor), and a *fixed namespace allow-list* (Plate.js). Only one of those five is capable of carrying a namespace it has never heard of. The thirteenth participant adds a **sixth** kind — and it is the only one that is not a vocabulary boundary at all: **Docxodus draws its line around a set of *parts*, not a set of *names*** (§3.12).
- **Five editors can carry a namespace they have never heard of: GenOffice, Folio, eigenpal, Docxodus, and — for whole parts — WordCraft.** Everything else drops unfamiliar markup at a boundary, and the corpus contains a namespace (`ns0` = `http://example.com/fixture`) that appears in **no** editor's allow-list, precisely to find that boundary.
- **The single sharpest probe in the corpus is `w16du:dateUtc`** — a single attribute, on a revision wrapper, in a namespace Microsoft introduced in 2023. **Eight of the twelve editors drop it**, including Microsoft Word's own round-trip baseline, LibreOffice. Four keep it — and the thirteenth participant, Docxodus, is the **only participant in the study that keeps it by decision rather than by accident**: it declares a typed `XName` for the attribute and validates its value (`RevisionOps.cs:37`, `:565`). **Not one of the twelve reports that it dropped it.**
- **Apryse WebViewer was measured, not read.** Its code is closed, so no claim in this report about *how* an editor serializes a namespace covers it, it has no column in the mechanism matrix (§6), and the counts in the prose are of the participants whose code was read. Its output was measured like every other: it keeps every name in 21 of the 28 namespaces, part of `w`, `a` and `w14`, and nothing of `cs`, `w16du`, `w15` and `w16cid`. It is one more editor that drops `w16du:dateUtc`. Its column is in the survival matrix (§4.2) and its rows in the partial losses (§4.3).

- **The measured results mostly agree with the static reading, and the disagreements are all in the same direction: the code understates the loss.** WordCraft's code has no `w16du` handling — and it also loses `w15` and `w16cid`, which the code neither mentions. Plate.js's `PASSIVE_NAMESPACES` list says `w14` is passive — and the measurement shows `w14:paraId` and `w14:textId` are lost anyway, because the loss happens in the HTML step *before* the passive list is consulted.
- **The `mc` namespace is where editors visibly disagree, and the disagreement is about which branch to take.** Word's semantics say: pick the `mc:Choice` whose `Requires` you support. Mammoth-based editors (Plate.js, docs.plus) take the **`Fallback`** — the opposite branch — and the reader sees *different words*. Folio takes the **first Choice without evaluating `Requires`** when the node is edited, and keeps the construct verbatim when it is not. eigenpal evaluates `Requires` properly. **Seven editors keep both branches; three keep only the `mc:Ignorable` attribute; two lose the namespace entirely.** The corpus's two branches both carry visible text, so a wrong branch choice is visible on the page, not just in the file.
- **Plate.js is the only editor that loses the image.** Its `pic:` namespace goes from 6 distinct names to **0**, and its `resource-omitted` diagnostic says why: *"An embedded image was omitted during DOCX import."* Every other editor that keeps a drawing keeps the picture.
- **Losing a namespace is almost never loud.** Of the twelve editors, exactly **two** have a namespace-level reporting mechanism: Plate.js (38 typed diagnostics, `lossPolicy` defaults to `'reject'`) and SuperDoc (a named diagnostic code per fail-closed drawing family). The other ten drop whole vocabularies without a word. **Docxodus is the thirteenth participant, and at the serialization layer it drops nothing** — which is exactly why the obligation moves one layer up, to its renderer, which names what it cannot draw (`[SMARTART]`, `[MATH]`).

---

1. Method

1.1 What this report measures

Serialization is one of three axes that the first report compressed into a single number. The three are:

1. **Serialization** — the editor held the construct in a model and wrote it back out. It understood it, at least partially, and could have changed it.
2. **Pass-through** — the editor never looked at the bytes. It copied them because it had no reason to touch them.
3. **Rendering** — the editor drew something on screen.

This report is entirely about the first axis, and it looks at it **per namespace** rather than per document. The companion report *Pass-Through* covers the second axis; the companion report *Rendering Versus Word* covers the third.

The unit of analysis is deliberately **the namespace, and within it the distinct local name** — not the element count. The reason is that element counts lie: a regenerating writer can emit *more* `w:` elements than the source and still have lost every revision wrapper. WordCraft's output contains **more `w:`** elements than the corpus and keeps **294 of 309** distinct names. A count-only measure would call that "kept". A distinct-name measure says what actually survived.

1.2 Two kinds of evidence, and why both are needed

Every claim in §5 rests on two independent bodies of evidence, kept in separate sections so they can be checked against each other:

Evidence	How it was obtained	What it supports	What it cannot support
Static code evidence (§3)	Every editor's save and import path read at its pinned commit in `repos/`. Every claim carries a `file:line`.	What the editor *intended* to do with a namespace, and where its boundary is.	What the editor *actually did* — code paths can be unreachable, and they can be wrong.
Measured survival (§4)	The corpus round-tripped through every editor, then every namespace compared **by URI**, as a set of distinct local names.	What the editor *did*. Reproducible by anyone.	Why. The measurement cannot distinguish a deliberate drop from an accident.

The two disagree in four places among the twelve editors (§4.4), and in every one of them the **code reading understates the loss**. §4.5 shows the opposite happening too — and with it the reason the disagreement matters: a boundary that is defined for the *modeled* namespaces says nothing about what happens to markup that was destroyed upstream of the boundary, and just as little about markup that survived by a route the reading never saw.

1.3 The corpus

The same 35-part adversarial corpus (`fixture/complex.docx`), corrected so that Microsoft Word 16.115 opens it. It contains a chart, four SmartArt parts, an OLE object with a real embedded `.docx`, an embedded workbook, an image, two tables (nested, merged, header rows, vertical merge), tracked insertions/deletions/runs/paragraph-marks/moves, content controls (including data-bound, date-picker and nested), complex and simple fields, bookmarks, footnotes, endnotes, comments, OMML math, `mc:AlternateContent` with both branches, a `w:altChunk`, `w:permStart`/`w:permEnd`, `w:proofErr`, `w14:*` attributes and elements, `w16du:dateUtc`, `w16cid:durableId`, bidi and RTL runs, and three sections including a landscape one with page borders.

One structural note about the corpus that matters for every namespace claim below: **`wp14` is declared.** It carries `mc:Ignorable="w14 w15 w16du w16cex w16cid wp14"` on 14 parts, and each of those 14 parts also carries `xmlns:wp14="http://schemas.microsoft.com/office/word/2010/wordprocessingDrawing"`. A consumer that resolves `mc:Ignorable` prefixes can resolve `wp14`. The adversarial foreign namespace in the corpus is `ns0`, bound to `http://example.com/fixture`.

---

2. The namespace inventory

2.1 What each namespace is

Counts are element uses and attribute uses across every XML and `.rels` part of the corpus, with the number of parts the namespace appears in. "prefix" is the conventional prefix; where a URI can be bound to more than one prefix, the alternatives are noted.

Prefix	Namespace URI	El	At	Parts	What it is in WordprocessingML
`w`	`.../wordprocessingml/2006/main`	882	627	14	**The main vocabulary.** Paragraphs, runs, properties, tables, numbering, styles, headers, footers, notes, comments, fields, content controls, revision markup. 309 distinct local names.
`a`	`.../drawingml/2006/main`	128	0	6	DrawingML main: geometry, fills, text bodies, theme elements. 60 distinct names.
`dgm`	`.../drawingml/2006/diagram`	69	0	5	**SmartArt**: the diagram data model, layout definition, colours and quick-style. 35 distinct names across four parts.
`c`	`.../drawingml/2006/chart`	67	0	2	The chart part: axes, series, categories, caches. 40 distinct names.
`ct`	`.../package/2006/content-types`	33	0	1	The content-types map: part → MIME type, plus the defaults for `.rels` and `.xml`.
`rels`	`.../package/2006/relationships`	33	0	4	The four `.rels` parts: every part's link to its neighbours.
`wp`	`.../drawingml/2006/wordprocessingDrawing`	20	0	1	The WordprocessingML wrapper around a drawing: `inline`/`anchor`, `extent`, `docPr`. 5 names.
`xml`	`w3.org/XML/1998/namespace`	0	17	5	`xml:space="preserve"` — 17 attributes, 0 elements. The smallest namespace and the one most often dropped by accident.
`r`	`.../officeDocument/2006/relationships`	0	17	1	The relationship **ids**: `r:id`, `r:embed`, `r:dm`, `r:lo`, `r:qs`, `r:cs`. Six names, seventeen uses, and every one of them is a pointer into the package.
`mc`	`.../markup-compatibility/2006`	3	14	14	**Markup Compatibility.** `mc:AlternateContent`, `mc:Choice`, `mc:Fallback`, and the

Prefix	Namespace URI	El	At	Parts	What it is in WordprocessingML
					`mc:Ignorable` attribute that appears on 14 parts. Four distinct names.
w14	.../office/word/2010/wordml	5	8	1	Word 2010: `w14:paraId`/`w14:textId` (paragraph identity), `w14:textOutline`, `w14:solidFill`, `w14:prstDash`, `w14:round`. 12 names.
app	.../officeDocument/2006/extended-properties	9	0	1	`docProps/app.xml`: application, template, total time, page/word counts, company.
cs	.../office/drawing/2012/chartStyle	8	0	2	The chart-style parts (`style1.xml`, `colors1.xml`) — the chart's *appearance* companions.
pic	.../drawingml/2006/picture	6	0	1	The picture: `pic:pic`, `pic:blipFill`, `pic:nvPicPr`, `pic:spPr`.
wps	.../office/word/2010/wordprocessingShape	5	0	1	The Word 2010 shape: `wps:wsp`, `wps:txbx`, `wps:bodyPr`. The *modern* branch of the corpus's `mc:AlternateContent`.
cp	.../package/2006/metadata/core-properties	5	0	1	Core properties: title, creator, created, modified, last modified by.
ds	.../officeDocument/2006/customXml	3	2	1	The custom-XML datastore: `ds:datastoreItem`, `ds:schemaRefs`.
dc	purl.org/dc/elements/1.1/	4	0	1	Dublin Core: title, subject, creator, description.
m	.../officeDocument/2006/math	4	0	1	**OMML math**: `m:oMath`, `m:oMathPara`, `m:r`, `m:t`.
v	urn:schemas-microsoft-com:vml	4	0	1	**VML**, the legacy drawing format: `v:shape`, `v:imagedata`, `v:textbox`. The *fallback* branch of the corpus's `mc:AlternateContent`.
w16du	.../office/word/2023/wordml/word16du	0	4	2	**Word 2023 UTC dates**: `w16du:dateUtc`, an attribute on revision wrappers. One name, four uses, two parts.
o	urn:schemas-microsoft-com:office:office	1	1	1	**OLE**: `o:OLEObject` and its `o:title`.
dcterms	purl.org/dc/terms/	2	0	1	Dublin Core terms: `dcterms:created`, `dcterms:modified`.
xsi	w3.org/2001/XMLSchema-instance	0	2	1	`xsi:type` on the chart and datastore properties.
ns0	http://example.com/fixture	2	0	1	**The adversarial foreign namespace.** Two elements in `customXml/item1.xml`. Appears in no editor's allow-list.
w10	urn:schemas-microsoft-com:office:word	1	0	1	`w10:wrap` — VML text wrapping.
w15	.../office/word/2012/wordml	0	1	1	`w15:restartNumberingAfterBreak`. One attribute.
w16cid	.../office/word/2016/wordml/cid	0	1	1	`w16cid:durableId` — the durable id of a comment. One attribute.

Prefix	Namespace URI	El	At	Parts	What it is in WordprocessingML
w16cex	.../office/word/2018/wordml/cex	0	0	14	Comment extensions. **Declared and mc:Ignorable-listed on 14 parts, never actually used.** A weak probe — flagged as such rather than counted as a test case.

Two namespaces are declared-but-empty and are excluded from the survival matrix: w16cex (above) and wp14. wp14 is declared as xmlns:wp14 on the same 14 parts that list it in mc:Ignorable, and the corpus does not use it as an element or an attribute.

2.2 Four traps built into the corpus

These are the details that make the corpus discriminating rather than merely large.

Trap 1 — the ignorable prefix is declared. mc:Ignorable="w14 w15 w16du w16cex w16cid wp14" names six prefixes, and all six are declared, including xmlns:wp14. The prefix that is foreign to WordprocessingML is ns0 (http://example.com/fixture) in customXml/item1.xml. Any editor that reconstructs the mc:Ignorable attribute from a canonical list (rather than copying it) will silently change it — and mc:Ignorable is one of the four mc names the measurement tracks, which is why Fidus, canvas-editor and WordCraft show mc at 1/4 rather than 0/4: they keep the *attribute* and lose the *structure*.

Trap 2 — w16du:dateUtc is an attribute on a revision wrapper. It is not an element; it is w16du:dateUtc on w:ins and w:moveTo. An editor can lose the whole w:ins element and keep the namespace score, or keep the w:ins element and lose the attribute. The measurement counts it as a *name* in the namespace's distinct set (@dateUtc), so it is tracked independently of whether the wrapper survives.

Trap 3 — the mc:AlternateContent sits inside a w:r, and both branches carry visible text. mc:Choice Requires="wps" holds a wps text box; mc:Fallback holds a v:shape/w:pict. The Fallback's text is *"AlternateContent fallback text"* and the Choice's is *"Text box content"*. **A wrong branch choice is visible on the page.** This is what makes mc the most consequential namespace in the corpus, and it is why the mc row in §4.2 is worth more attention than its single-element count would suggest.

Trap 4 — the foreign namespace. customXml/item1.xml uses ns0 = http://example.com/fixture. No editor's namespace table, allow-list, or dispatch switch contains it. It is the cleanest possible test of whether an editor's boundary is **open** (anything unrecognised is carried) or **closed** (anything unrecognised is discarded).

---

3. Where each editor's serialization boundary is

This section is the static half. Each subsection states the boundary *mechanism* first, because the mechanism predicts the behaviour far better than the editor's name does.

3.1 SuperDoc — a proprietary engine with an open contract layer

SuperDoc's DOCX engine is not auditable from source. The repository depends on @superdoc/docx-engine 0.28.0-next.3, whose licence forbids deobfuscation; its parsing lives in a minified dist/ bundle. Every claim below is about the **open contract layer** around the engine, and the boundary that layer draws is a *drawing taxonomy*, not a namespace table.

SuperDoc classifies drawings into three support levels — supported, **fail-closed**, deferred (packages/layout-engine/contracts/src/drawing-taxonomy.ts:33-42):

Family	Level	Evidence	Consequence
bitmap / vector shape families	supported	drawing-taxonomy.ts:213-240	drawn
mc:AlternateContent	supported, fail-closed on the branch that cannot be rendered	:255-256, :372-373, :376	payload preserved for save/reopen

Family	Level	Evidence	Consequence
		(`altContentNoSupportedChoice`)	
chart (**C**)	supported, **fidelity deferred**	`:246-247`, `:400-401`	renders, not round-tripped exactly
SmartArt (`dgm`)	**fail-closed**	`:328-329`, `:332`	preserved, placeholder drawn, diagnostic emitted
VML (**V**)	**fail-closed**	`:335-336`, `:342-343`, `:348`, `:406-407`	preserved, placeholder drawn
OLE (**O**)	**fail-closed**	`:321-322`, `:325`	preserved, placeholder drawn

The one SuperDoc namespace that is **load-bearing rather than merely preserved** is `w14`: `w14:paraId` / `w14:textId` are the *identity substrate* of the whole model — the paragraph identifier that comments and lists hang off (`packages/document-api/src/types/sd-nodes.ts:45-47`; `packages/document-api/src/contract/schemas.ts:1237`). `w14:ligatures`, `w14:numForm`, `w14:numSpacing`, `w14:stylisticSets` and `w14:cntxtAlts` are addressable format attributes (`packages/document-api/src/format/inline-run-patch.ts:388-392`).

The gap, stated plainly: `w15`, `w16cid`, `w16cex`, `w16du` and `m` have **no verifiable evidence** in the open repository. Their behaviour is inside the closed engine. That is an absence of public evidence, **not** evidence of absence — and §4.2 replaces it with a measurement.

3.2 Folio — a total dispatch table over declared children, with a verbatim sink

Folio is the most explicitly documented boundary in the set, and it is the one design that comes closest to being lossless without being byte-exact.

Its **part policy** is a file (`packages/core/src/docx/packageParts.ts:6-30`): modelled parts are rewritten from the model; embedded objects, media, custom XML, fonts, charts, macro projects and unknown parts are **handed back byte for byte**.

Its **element dispatch** is a *total map over the declared children of each container*, compiler-enforced — a container cannot gain a declared child without somebody choosing `CAPTURE`, `ownedElsewhere` or a handler (`packages/core/src/docx/containerChildren.ts:9-14`; `containerChildren.gen.ts`, generated from the committed schema graph). Everything undeclared falls into an **ordered verbatim sink** (`containerChildren.ts:1-29`, `:47-48` (CAPTURE), `:342-346`), whose own comment says it exists because *"a name the schema does not know is exactly the case a hand-written `default` gets wrong."*

The `mc` policy is stated in Folio's own source, at `packages/core/src/docx/xmlParser.ts:571-594`:

> Folio's normalized Markup Compatibility policy: use the first Choice, which is the established parser behavior when `mc:Requires` cannot be evaluated; use Fallback only when there is no Choice. Callers serialize that selected branch as ordinary WordprocessingML rather than concatenating alternatives.

So Folio **flattens** `mc:AlternateContent` to its chosen branch and loses the structure — but keeps the content. Its handling of the other namespaces is explicit: `w14:paraId`/`w14:textId` are read and written as paragraph identity, with prefixes matched **by URI, not by literal prefix** (`packages/core/src/docx/paraIdAttribute.ts`; `packages/core/src/docx/wordprocessingPrefixes.ts:153` (`W14_URI`)); revision wrappers inside an *unmodelled* carrier are kept verbatim, **counted**, and reported as `revisionCarrierOpaque` (`packages/core/src/docx/parser.ts:369-394`, `:957-979`; `parseWarningMessage.ts:87-89`); `w:altChunk` is reported as `altChunkUnsupported` (`parser.ts:952`); charts and diagrams are modelled only as a **bounded summary** (`chartPreview.ts`, `diagramPreview.ts`) while their parts are returned byte for byte; OMML is converted to MathML (`mathToMathml.ts`).

Folio's boundary is open, in the sense that matters: an undeclared element is not discarded, it is carried.

3.3 GenOffice — a byte-range splice that never re-serializes

GenOffice's engine states the policy in its own source (`packages/docx-engine/src/scan.ts:1-9`):

> We never re-serialize the whole XML (parse->serialize would silently change untouched bytes: attribute order, self-closing forms, entity forms…). Instead we locate the exact character range of every top-level element in `<w:body>` so that patch-save can splice new fragments while copying untouched elements as raw original substrings.

The splice is literal (`packages/docx-engine/src/patch.ts:1644-1648`):

```
if (fb.kind === 'original') {
  const el = elements[fb.docxIndex]
  if (!el) throw new Error(`invalid docxIndex ${fb.docxIndex}`)
  xml = documentXml.slice(el.start, el.end)
```

Every part that is not explicitly patched is copied byte-for-byte from the source zip (`patch.ts:2133-2135`), and `opaqueRegions` — spans the model did not take — are re-emitted as raw source slices (`patch.ts:1564-1569`; `parse.ts:341`, `:685`).

What this means for namespaces is precise and unusual: **GenOffice has a canonical prefix table of 19 URIs** (`packages/docx-engine/src/ooxml-normalize.ts:50-75`) that it matches *literally* — `w`, `r`, `m`, `a`, `wp`, `pic`, `c`, `dgm`, `lc`, `mc`, `wps`, `wpg`, `wpc`, `w14`, `w15`, `v`, `o`, `w10` — matching "by URI + localName, never by localName alone, so unrelated namespaces that happen to reuse a local name are left untouched." But the table only governs **renaming**. `w16du`, `w16cid`, `w16cex`, `wp14` and `ns0` are **not in it**, and they survive anyway, because the bytes are spliced rather than re-serialized. **GenOffice's boundary is open by construction**: a namespace the engine has never heard of is carried, because carrying it is cheaper than modelling it.

Two GenOffice details do cost something. `mc:Choice` is authoritative for display, and the Fallback "re-emits the same picture … doubling the drawn image" — the Fallback is skipped only when a Choice in the same block already produced an image (`packages/docx-engine/src/parse.ts:5915-5934`). And the revision engine is a **raw-XML accept/reject** engine operating on source markup, opt-in and not applied by default (`packages/docx-engine/src/revision-xml.ts:105-112`).

3.4 eigenpal — a typed vocabulary plus a generic catch-all

eigenpal is the only editor whose model has an explicit *fallback node kind* for content it does not understand, and the source says why (`packages/core/src/store/package/ooxml-tree.ts:1252-1261`):

> Any element the typed vocabulary does not cover — the other half of the preservation model.
>
> Also where a KNOWN element lands when it appears somewhere invalid: an element is demoted to generic rather than rejected, so malformed or unfamiliar content is carried losslessly and never locks editing.

Two consequences follow, and both are measurable. First, **an unknown namespace is not lost** — it is carried as a `generic` node. Second, `mc:AlternateContent` is recognised **by namespace URI + localName**, not by prefix: `node.kind === 'generic' && node.namespaceUri === MC_NAMESPACE_URI && node.localName === 'AlternateContent'` (`packages/core/src/store/package/drawing-projection-walk.ts:50-70`).

eigenpal is also the only editor that evaluates `mc:Requires` **properly**: branch selection honours it against a supported-namespace set (`DEFAULT_SUPPORTED_MC_REQUIRES`, `packages/core/src/store/package/drawing-projection.ts:312`; the selector itself at `:549`, `selectCompatibilityBranch`; the run-level variant at `:1291-1340`). Unsupported drawing kinds raise named issues (`drawing-support.ts`; `drawing-support-issues.ts`), and the export layer emits **typed warnings** for lossy conversions with a closed code set (`packages/core/src/export/export-content-warnings.ts:1-45`: `legacy-textbox`, `legacy-drawing`, `scan-limit`, `unsupported-style-separator`).

eigenpal's boundary is open and it says so. This is the design closest to "you cannot lose a construct by accident".

3.5 office-open — a prefix-literal switch with a silent `default`

office-open is a library: `packages/docx/src/parse.ts:501` parses, `packages/docx/src/generate.ts:71` generates.

Its XML layer is genuinely namespace-aware — `normalizeNamespaces` maps *namespace URI → canonical prefix*, so "a source that binds the Word namespace to `ns0:` parses into `w:` names" (`packages/xml/src/types.ts:78-92`; `packages/xml/src/parse.ts:286-295`, `:490-507`), wired at `packages/core/src/opc/parser.ts:27`. The canonical prefix table covers 28 URIs including `w14`, `w15`, `w16cid`, `w16cex`, `w16sdtdh`, `w16se`, `wp14`, `wpc`, `wpg`, `wpi`, `wps`, `dsp`, `cx` (`packages/core/src/opc/namespaces.ts:196-245`).

But the **model** is not namespace-aware, and that is where the loss happens. The paragraph-child dispatch is a `switch` on **literal prefixed names** (`packages/docx/src/body.ts:2958-3114`), and its `default:` branch is `break` — i.e. **silently dropped**. Two namespaces are absent from the canonical table entirely, and one of them is the corpus's sharpest probe:

- **`w16du`** — its URI is **not in `packages/core/src/opc/namespaces.ts` at all**. "Unknown URIs keep their source prefix verbatim" (`packages/xml/src/types.ts:88-89`), and an unrecognised attribute is not carried into the model. **`w16du:dateUtc` is dropped, with no diagnostic.**
- **`mc`** — no `mc:AlternateContent` branch selector was found in the DOCX layer, so `mc:` content at the paragraph-child level would reach the `default:` branch. **The measurement shows otherwise** (§4.4, disagreement 5): the construct survives in office-open's and Docen's output with both branches intact, most likely because it reaches the model as an opaque `unsupportedDrawing` payload rather than through the paragraph-child switch.

office-open does have an escape hatch at the *drawing* level: a drawing the parser cannot model becomes an `unsupportedDrawing` node carrying the **raw XML**, which is re-emitted on save (`packages/docx/src/parts/drawing/drawing-parse.ts:225-284`; `packages/docx/src/parts/paragraph/run/unsupported-drawing-run.ts:163`, `stringifyUnsupportedDrawing`; call site `packages/docx/src/parts/inline-drawings.ts:304`). `wps` is recognised by URI (`drawing-parse.ts:158`).

office-open's boundary is closed at the paragraph-child level and open at the drawing level — and the two levels are not consistent with each other, which is the single most surprising thing about its design. The `mc:AlternateContent` is a concrete case: it sits inside a `w:r`, contains a drawing, and **survives** — by the drawing-level escape hatch rather than by the paragraph-level model.

3.6 Docen — a disposition table over office-open's model

Docen consumes office-open's model and adds a per-kind **disposition table** over extension kinds (`packages/docx/src/extensions/coverage.ts:1-140`): each kind is marked `modeled`, `passthrough`, `inlinePassthrough`, `opaque` or `dropped` — e.g. `unsupportedDrawing: { passthrough: "inlinePassthrough atom" }` (`coverage.ts:138`). Passthrough payloads are carried as opaque node trees and re-emitted unchanged (`packages/docx/src/extensions/passthrough.ts:1-80`; `group-members.ts:45`).

Docen adds one thing office-open does not have: `w14:rPr` raw XML is preserved as a **round-trip-only marker attribute** — "Round-trip-only markers: raw w14:rPr XML and a bare `<w:rPr/>`" (`packages/docx/src/extensions/text-style.ts:108-109`, `w14RawXml: attrNative()`).

What Docen does **not** add is any handling for `mc`, `w16du`, `dgm` or `c` — those inherit office-open's boundary and are dropped at the paragraph-child `default:`.

3.7 WordCraft — a typed Rust story model that regenerates what it models

WordCraft is the only Rust editor here, and the only one with a real namespace-constant table (`crates/docx/src/smart_art.rs:25-28`: `NS_DGM`, `NS_DSP`, `NS_A`, `NS_R`; `crates/docx/src/chart_spec.rs:15-17`: `NS_C`, `NS_A`, `NS_R`).

Its `mc` handling picks the **first Choice the reader recognises**, from a literal list of `Requires` prefixes (`crates/docx/src/read/story.rs:161-175`; the known list at `:164`; used from `read/story/blocks.rs:62`). Its `dgm` handling is a dedicated SmartArt reader **and** a SmartArt writer (`crates/docx/src/read/diagram.rs`) — the diagram is **regenerated from a model, not copied**, which is a different act from preservation and is the reason WordCraft's SmartArt renders differently from the original.

Its `c` handling is the opposite: the chart part itself is **not supported**, and an explicit in-content placeholder is emitted (`crates/docx/src/read/chart.rs:137`, "not supported yet"). `w16cex:dateUtc` **is** read from the comment-ex parts (`crates/docx/src/read/meta.rs:272`). `w14:contentPart` is listed among the relationship-bearing elements (`crates/docx/src/xml.rs:140-151`, `REL_ELEMENTS`).

And there is no `w16du`, no `w15`, no `w16cid` anywhere in `crates/docx/src` — no read table, no write table. Those three attributes vanish on write, and the code does not mention them at all.

3.8 Plate.js — a closed passive vocabulary

Plate.js's DOCX **import** is Mammoth (DOCX → HTML); its DOCX **export** is HTML → DOCX — a full regeneration — with a source-preservation lane that may copy eligible parts forward from the original file.

The import boundary is Mammoth's **9-entry namespace map** (`mammoth/lib/docx/office-xml-reader.js:8-32`): `w`, `r`, `wp`, `a`, `pic`, `mc`, `v`, `office-word`, and the Word-2010 URI — the last of which is bound to the prefix **`wordml`, not `w14`**. Any URI not in that map becomes a Clark-notation name `{uri}localName` and can therefore never match a reader (`mammoth/lib/xml/reader.js:53-67`, `convertName`). Unmatched elements are reported: `warning("An unrecognised element was ignored: " + element.name)` (`mammoth/lib/docx/body-reader.js:48-53`) — **except** for a hard-coded silently-ignored list including `w:moveFrom` and `w:del` (`body-reader.js:718-739`, `ignoreElements`), which bypasses the warning entirely.

Mammoth's `mc` handling is the sharpest divergence from Word in the whole study: `collapseAlternateContent` replaces `mc:AlternateContent` with **the `mc:Fallback` children** (`mammoth/lib/docx/office-xml-reader.js:39-42`). Mammoth picks the *opposite branch* from the one Word picks.

The export boundary is Plate's own, and it is the only **closed namespace allow-list** in the study (`packages/platejs/src/docx/internal/sourceEligibility.ts:66-95`):

- `w14`, `w15`, `w16cid`, `w16cex`, `wps`, `mc`, `m`, `a`, `pic`, `wp`, `v`, `o`, `w10` — **in** `PASSIVE_NAMESPACES`, so markup in these namespaces may be carried forward verbatim.
- `w16du`, `dgm`, `c`, `ns0` — **not** in it, so a part containing them fails eligibility with the violation feature `'markup'` and the detail `"contains unrecognized markup"` (`sourceEligibility.ts:21`, `:38`, `:359`, `:412`), and the part is regenerated instead.

On this corpus Plate's gate refuses the byte-exact path outright, and says why:

> `source-unavailable`: The DOCX source cannot be retained because `word/_rels/document.xml.rels` declares an unrecognized relationship (chart); export will generate the document from editor content.

3.9 Fidus Writer — a literal-prefix if/else chain

Fidus is Django + ProseMirror, and its DOCX importer parses with `fast-xml-parser` — **not namespace-aware**; prefixes are literal tag names (`document/static/js/modules/exporter/tools/xml.js:1-16`: `attributeNamePrefix: ""`, `preserveOrder: true`).

The importer's paragraph-child handling is an if/else chain over `child.tagName` handling `w:r`, `w:fldChar`, `w:footnoteReference`, `w:endnoteReference`, `w:commentRangeStart/End`, `w:hyperlink`, `m:oMath`, `w:sdt` (`document/static/js/modules/importer/docx/convert.js:930-1098`). **Everything else** — including `w:drawing`, `w:pict`, `w:object`, `mc:AlternateContent`, `dgm:*`, `c:chart` — falls through to:

```
console.warn("unhandled node", child)
```

Two further limits: only the **first** `w:t` in a run is taken (`child.query("w:t")`, `convert.js:1042`), and tracked changes are recovered *positionally* via `child.closest("w:ins")` / `closest("w:del")` and turned into ProseMirror marks rather than a revision tree (`convert.js:1043-1046`). There is **no `mc:AlternateContent` handling anywhere** in Fidus's importer or exporter.

Fidus's boundary is the narrowest in the study. It is a chain of string comparisons, and everything not named in the chain is dropped with a warning that never reaches the user.

3.10 canvas-editor + canvas-editor-plugin — a prefix-literal DOM dispatch with no `else`

canvas-editor itself contains **no DOCX code at all** — all of it lives in the separate canvas-editor-plugin checkout. The DOM parser is prefix-literal (`canvas-editor-plugin/packages/docx/src/docx/importDocx.ts:876`, `:885`, `:888`): `child.tagName === 'w:p'`, `child.tagName === 'w:tbl'`, `child.tagName === 'w:sdt'`.

The paragraph-inline loop is an if/else chain on `const tag = child.tagName` **with no `else`** (`importDocx.ts:1271-1281`, `:1394`), so unmatched children are dropped with **no warning and no diagnostic**. `parseDrawing` extracts **only** `a:blip` (bitmap) or `v:imagedata` (VML image); a SmartArt or chart drawing yields `null` and is dropped silently (`importDocx.ts:1578-1590`). `mc:AlternateContent` is not handled at all — the whole text box, both branches, is dropped.

The only extension namespace canvas touches is `w14`, and only one corner of it: `w14:checkbox` and `w14:checked` on content controls (`importDocx.ts:1421`, `:1436`, `:1440-1441`). VML is handled only as a watermark (`v:shape` with `v:textpath`/`v:imagedata`) and as a horizontal-rule separator (`v:line`, flat `v:rect`) (`importDocx.ts:782-820`, `:1199-1200`).

3.11 docs.plus — Mammoth again, but the reporting layer is better

docs.plus's DOCX import is Mammoth — the **same 1.13.0 build** as Plate's (`bun.lock:117` → `mammoth@1.13.0`) — so its namespace map and its Fallback-branch behaviour are identical to §3.8's.

What differs is the reporting layer. Mammoth's messages are mapped to **`ConversionWarning`** objects with codes `media-placeholder-dropped`, `title-promoted-paragraph`, `title-synthesized`, `unsupported-element` (`apps/hocuspocus.server/src/modules/document-conversion/domain/docxImport.ts`; type at `apps/hocuspocus.server/src/modules/document-conversion/types.ts:13-29`), and `ImportResult` carries `warnings: ConversionWarning[]` back to the caller (`types.ts:25-29`). The stated contract is explicit:

> Lossy conversions are reported, never silent — the user has to know what changed. (`types.ts:19`)

docs.plus therefore has the same XML fidelity as Plate.js and a materially better report of it — which is a useful control: it shows that the Mammoth boundary and the *reporting* boundary are separable decisions.

3.12 Docxodus — an OpenXML SDK that draws its boundary around *parts*, not around vocabularies

Docxodus is not an editor, and that is the point of putting it in this table. It is a .NET OOXML engine — the `docxodus` npm package (15.0.0) with the companion `@docxodus/export` (15.0.0), built on the `DocumentFormat.OpenXml` SDK 3.5.1 — published as WebAssembly. Its product surface is a *SDK*: an in-browser block editor, a paginated DOCX→HTML materializer, and a package writer. It enters this study twice: as the **package writer** measured in §4, and as the **renderer** measured in *Rendering Versus Word*. What follows is the write path only.

The boundary mechanism is a list of parts. The other five mechanisms in this report all draw a line around a *vocabulary* — a set of element names to dispatch on. Docxodus draws it around a set of *files*. `Docxodus/DocxSession.cs:968-986` — `EnumerateProjectedParts` yields the main document part, every header and footer part, footnotes, endnotes, the comments part and its three companions (`commentsExtended`, `commentsIds`, `people`), the settings part, the styles part, and the custom-XML parts. Those are the only parts the projector can write to.

Everything outside that list is carried byte-for-byte. The package is a ZIP and the SDK writes it entry by entry; `Docxodus/ZipPackageOutputNormalizer.cs:14` states the rule in the source's own words: *"The package payload is copied entry-for-entry; only ZIP compression and extraction metadata are normalized."* The chart, the four SmartArt parts, the image, the OLE object, the embedded workbook and the custom-XML datastore are none of them on the projected list, so none of them is re-written.

The XML manipulation is LINQ-to-XML, not a typed DOM. This is the single fact that explains the rest. The save path flushes each projected part's cached `XDocument` back into its part and then saves the package (`Docxodus/DocxSession.cs:838-842`; the `GetXDocument`/`PutXDocument` accessors are at `Docxodus/PtOpenXmlUtil.cs:141`). **`OpenXmlUnknownElement` appears nowhere in the repository** — so the SDK's schema model is never the thing that decides whether an element survives. An element Docxodus has never heard of is simply a node in an `XDocument`, and nodes in an `XDocument` are carried.

Only the parts the projector actually materialised are re-written. `DocxSession.cs:838-840` writes a part back only when `part.GetXDocument().Root is not null` — that is, only when the projector opened it. The measurement matches exactly: **10 of the corpus's 35 parts come back re-serialised and 25 come back byte-identical.** The re-serialised set is `document.xml`, `comments.xml`, `footnotes.xml`, `endnotes.xml`, `header1.xml`, `header2.xml`, `footer1.xml`, `footer2.xml`, `settings.xml` and `styles.xml`. `customXml/item1.xml` is a *candidate* for projection but comes back byte-identical, because the projector never opened it. **This is the rule that derives §6's Docxodus column**, and it is stated here rather than left to be reverse-engineered from the cells.

`w16du` is the one namespace Docxodus handles by name. `Docxodus/Internal/RevisionOps.cs:37` declares a typed `XName` for `w16du:dateUtc`, and `RevisionOps.cs:565` rejects a malformed value with an `invalid_revision_date_utc` diagnostic. The corpus's attribute is one of the values the product's own test suite pins (`Docxodus.Tests/DocxSessionBulkRevisionSafetyTests.cs:40`). **This is the first participant in the study whose `w16du` keep is a decision rather than a by-product** — the other four keepers keep it because their architecture carries it for free.

And then there is a second path, one layer over, where the same product drops it. The IR diff renderer normalises a *preserved original clone* and, in the course of it, strips *"Non-W-namespace extension attributes on revision elements (e.g. Word 2023's `w16du:dateUtc`)"*, on the stated ground that *"the extension attrs are undeclared noise to the SDK schema the output is validated against"* (`Docxodus/Ir/Diff/IrMarkupRenderer.cs:10795`). **So the same product keeps `w16du:dateUtc` in the DOCX package and removes it in the diff/merge renderer.** It is the package-versus-render distinction in its sharpest possible form: one attribute, two layers, opposite answers.

What this means for the boundary taxonomy. A vocabulary boundary has a floor and a ceiling: names above the line are handled, names below it are dropped. Docxodus's boundary has only a floor — **inside a projected part there is no name the projector has to know, because the XML is a tree and not a dispatch table.** The consequence cuts both ways, and the report should say so plainly: Docxodus **cannot lose a namespace it has never heard of**, and it also **cannot edit one**. Its floor is the strongest in the study; its ceiling is the only one that is not an editor's ceiling at all, because it was never trying to be an editor.

---

4. Measured namespace survival

4.1 How the measurement works

`build/ns_map.py` opens every output package, walks every XML and `.rels` part, and records, **per namespace URI**, the set of **distinct local names** used — elements as `name`, attributes as `@name`.

Two decisions matter:

1. **Namespaces are matched by URI, never by prefix.** Mammoth binds the Word-2010 URI to the prefix `wordml` rather than `w14`; a prefix-based comparison would call that a loss. The URI-based comparison calls it a keep, which is correct — the namespace is the same vocabulary.
2. **The unit is the distinct name, not the count.** WordCraft emits *more* `w:` elements than the corpus and still loses 15 of the 309 distinct names. Only the name set answers "is this construct still there at all?".

The comparison is `kept = |corpus_names ∩ output_names|` over `|corpus_names|`. A namespace the corpus never uses is reported as `-`.

4.2 The survival matrix

`all` = every distinct name kept · `none` = the namespace is entirely absent · `k/n` = partially kept.

WordprocessingML and revision namespaces

Namespace	Corpus names	SuperDoc	Folio	GenOffice	eigenpal	office-open	Docen	WordCraft	Fidus	canvas	docs.plus	Plate.js	Docxodus	Apryse	LibreOffice
`w`	309	all	all	all	all	308/309	305/309	294/309	**98/309**	**119/309**	**114/309**	**108/309**	all	300/309	260/309
`r`	6	all	all	all	all	all	all	all	**1/6**	**2/6**	**2/6**	**none**	all	all	all
`xml`	1	all	all	all	all	all	all	all	all	all	all	all	all	all	all
`mc`	4	all	all	all	all	all	all	**1/4**	**1/4**	**1/4**	**none**	**none**	all	all	all
`w14`	12	all	10/12	all	all	10/12	10/12	all	**none**	**none**	**none**	**none**	all	10/12	10/12
`w15`	1	all	all	all	all	all	all	**none**	all	all	**none**	**none**	all	**none**	**none**
`w16cid`	1	all	all	all	all	**none**	**none**	all	**none**	**none**	**none**	**none**	all	**none**	**none**
`w16du`	1	all	all	all	all	**none**	**none**	**none**	**none**	**none**	**none**	**none**	all	**none**	**none**

Drawing, chart, diagram and legacy namespaces

Namespace	Corpus names	SuperDoc	Folio	GenOffice	eigenpal	office-open	Docen	WordCraft	Fidus	canvas	docs.plus	Plate.js	Docxodus	Apryse	LibreOffice
`a`	60	all	all	all	all	all	all	56/60	**11/60**	**11/60**	42/60	33/60	all	58/60	59/60
`wp`	5	all	all	all	all	all	all	all	3/5	all	4/5	**none**	all	all	all
`pic`	6	all	all	all	all	all	all	all	all	all	all	**none**	all	all	all
`dgm`	35	all	all	all	all	all	all	all	**none**	**none**	**none**	**none**	all	all	all
`c`	40	all	all	all	all	all	all	all	**none**	**none**	**none**	**none**	all	all	39/40
`cs`	8	all	all	all	all	all	all	**none**	**none**	**none**	**none**	**none**	all	**none**	**none**
`wps`	5	all	all	all	all	all	all	all	**none**	**none**	**none**	**none**	all	all	all
`v`	3	all	all	all	all	all	all	2/3	**none**	**none**	**none**	**none**	all	all	all
`o`	2	all	all	all	all	all	all	all	**none**	**none**	**none**	**none**	all	all	all
`w10`	1	all	all	all	all	all	all	**none**	**none**	**none**	**none**	**none**	all	all	**none**

Package, metadata and foreign namespaces

Namespace	Corpus names	SuperDoc	Folio	GenOffice	eigenpal	office-open	Docen	WordCraft	Fidus	canvas	docs.plus	Plate.js	Docxodus	Apryse	LibreOffice
`ct`	3	all	all	all	all	all	all	all	all	all	all	all	all	all	all
`rels`	2	all	all	all	all	all	all	all	all	all	all	all	all	all	all

Namespace	Corpus names	SuperDoc	Folio	GenOffice	eigenpal	office-open	Docen	WordCraft	Fidus	canvas	docs.plus	Plate.js	Docxodus	Apryse	LibreOffice
app	9	all	all	all	all	all	all	8/9	**3/9**	**1/9**	**none**	**none**	all	all	**4/9**
cp	5	all	all	all	all	all	all	all	**1/5**	3/5	4/5	4/5	all	all	all
dc	4	all	all	all	all	all	all	all	2/4	**1/4**	all	all	all	all	all
dcterms	2	all	all	all	all	all	all	all	all	all	all	all	all	all	all
ds	5	all	all	all	all	all	all	all	**none**	**none**	**none**	**none**	all	all	all
ns0	2	all	all	all	all	all	all	all	**none**	**none**	**none**	**none**	all	all	all
xsi	1	all	all	all	all	all	all	all	all	all	all	all	all	all	all

Reading the `w` row: **five participants keep every one of the 309 distinct `w:` names** — SuperDoc, Folio, GenOffice, eigenpal and Docxodus. office-open comes next, losing exactly one (`@dirty`), then Docen four and WordCraft fifteen; below them there is a cliff, where Fidus, canvas-editor, docs.plus and Plate.js each keep **roughly a third** of the main vocabulary.

4.3 Partial losses, name by name

The interesting losses are the partial ones, because they show exactly which constructs a boundary let through.

Namespace	Editor	Kept	Lost names
w	office-open	308/309	`@dirty`
w	Docen	305/309	`@dirty`, `cr`, `noBreakHyphen`, `softHyphen`
w	WordCraft	294/309	`@asciiTheme`, `@dirty`, `@first`, `@hAnsiTheme`, `@hint`, `@instr`, `@other`, `altChunk` … (+7 more)
w	Fidus	98/309	`@accent1`, `@accent2`, `@accent3`, `@accent4`, `@accent5`, `@accent6`, `@asciiTheme`, `@bg1` … (+203 more)
w	canvas	119/309	`@accent1`, `@accent2`, `@accent3`, `@accent4`, `@accent5`, `@accent6`, `@asciiTheme`, `@author` … (+182 more)
w	docs.plus	114/309	`@accent1`, `@accent2`, `@accent3`, `@accent4`, `@accent5`, `@accent6`, `@asciiTheme`, `@author` … (+187 more)
w	Plate.js	108/309	`@abstractNumId`, `@accent1`, `@accent2`, `@accent3`, `@accent4`, `@accent5`, `@accent6`, `@asciiTheme` … (+193 more)
w	Apryse	300/309	`@first`, `@orient`, `@other`, `altChunk`, `group`, `moveFromRangeEnd`, `moveToRangeEnd`, `paperSrc` … (+1 more)
w	LibreOffice	260/309	`@accent1`, `@accent2`, `@accent3`, `@accent4`, `@accent5`, `@accent6`, `@bg1`, `@bg2` … (+41 more)

Namespace	Editor	Kept	Lost names
r	Fidus	1/6	@cs, @dm, @id, @lo, @qs
r	canvas	2/6	@cs, @dm, @lo, @qs
r	docs.plus	2/6	@cs, @dm, @lo, @qs
mc	WordCraft	1/4	AlternateContent, Choice, Fallback
mc	Fidus	1/4	AlternateContent, Choice, Fallback
mc	canvas	1/4	AlternateContent, Choice, Fallback
w14	Folio	10/12	@paraId, @textId
w14	office-open	10/12	@paraId, @textId
w14	Docen	10/12	@paraId, @textId
w14	Apryse	10/12	@paraId, @textId
w14	LibreOffice	10/12	@paraId, @textId
a	WordCraft	56/60	defRPr, graphicFrameLocks, pPr, sysClr
a	Fidus	11/60	accent1, accent2, accent3, accent4, accent5, accent6, bgFillStyleLst, bodyPr … (+41 more)
a	canvas	11/60	accent1, accent2, accent3, accent4, accent5, accent6, bgFillStyleLst, bodyPr … (+41 more)
a	docs.plus	42/60	avLst, bodyPr, camera, defRPr, effectRef, endParaRPr, fillRef, fontRef … (+10 more)
a	Plate.js	33/60	avLst, blip, bodyPr, camera, defRPr, effectRef, endParaRPr, ext … (+19 more)
a	Apryse	58/60	defRPr, pPr
a	LibreOffice	59/60	sysClr
wp	Fidus	3/5	cNvGraphicFramePr, effectExtent
wp	docs.plus	4/5	cNvGraphicFramePr
c	LibreOffice	39/40	layout
v	WordCraft	2/3	textbox
app	WordCraft	8/9	AppVersion
app	Fidus	3/9	Company, DocSecurity, HyperlinksChanged, LinksUpToDate, ScaleCrop, SharedDoc
app	canvas	1/9	AppVersion, Application, Company, DocSecurity, HyperlinksChanged, LinksUpToDate, ScaleCrop, SharedDoc

Namespace	Editor	Kept	Lost names
app	LibreOffice	4/9	DocSecurity, HyperlinksChanged, LinksUpToDate, ScaleCrop, SharedDoc
cp	Fidus	1/5	category, keywords, lastModifiedBy, revision
cp	canvas	3/5	category, keywords
cp	docs.plus	4/5	category
cp	Plate.js	4/5	category
dc	Fidus	2/4	description, subject
dc	canvas	1/4	description, subject, title

Four details in that table are worth pulling out.

`@dirty` is lost by every editor that regenerates the `w` vocabulary. It is the attribute that tells Word "this field's cached result is stale, recalculate it." Losing it is not itself a data loss — but it is the reason a regenerated TOC or field shows the wrong thing until the user presses F9.

`w14:@paraId` and `w14:@textId` are lost by exactly four editors — Folio, office-open, Docen and LibreOffice — and by nobody else. WordCraft keeps them; so do SuperDoc, GenOffice and eigenpal. Folio's loss is *not* a namespace-boundary loss: Folio models `w14:paraId` as paragraph identity and writes it back. The mechanism is narrower and was found in the first report: the run parser applies the remainder attributes to `w:r`, but `parseRunProperties` reads only the *children* of `w:rPr` (`runParser.ts:696`) and never applies a remainder to it — so the attributes are **dropped, not deliberately removed**.

The `a` namespace tells the drawing story. Fidus and canvas-editor keep 11 of 60 `a:` names — and the ones they lose are the *theme* names (`themeElements`, `clrScheme`, `fontScheme`, `fmtScheme`, `effectStyleLst`, `fillStyleLst`, `bgFillStyleLst`) plus every text-body name (`bodyPr`, `p`, `r`, `t`, `rPr`, `pPr`). They keep the *geometry* names they need for a bitmap (`blipFill`, `spPr`, `prstGeom`, `xfrm`) and lose everything else, because they model a picture as a picture and nothing more. Plate.js keeps 33: it keeps the text-body names (`bodyPr`, `p`, `r`, `t`, `rPr`, `pPr`) because it has a text model, and loses `blip` — which is exactly the image loss the diagnostic reports.

The one participant with no row in this table is Docxodus, and its absence is the finding. Docxodus is the only participant in the study that loses **no** distinct name in any namespace — not one partial loss anywhere. Every other participant has at least one row. The reason is architectural rather than lucky: there is no name-based dispatch in its write path at all (§3.12), so there is no name that *can* be left behind.

4.4 Where the static reading and the measurement disagree

Four disagreements among the twelve editors, and **all four run the same way: the code understates the loss.** §4.5 shows the opposite happens as well.

Disagreement 1 — WordCraft and the namespaces the code never mentions. §3.7 showed that WordCraft has no `w16du` handling anywhere in `crates/docx/src`. The measurement adds three more losses: WordCraft also loses **`w15:restartNumberingAfterBreak`**, **`cs`** (the chart-style parts) and **`w10:wrap`**, and it loses **`v:textbox`** while keeping the other two `v` names. **It does not lose `w16cid:durableId`** — an earlier edition of this report said it did, and that was wrong; the attribute is present in `word/commentsIds.xml` in WordCraft's output, and §6 now carries the measured value. The code reading is right that there is no handler; it is incomplete because *absence of a handler for `w16du` implies absence of handlers for the neighbouring namespaces too* — though not for every one of them.

Disagreement 2 — Plate.js's `w14` is passive, and `w14:paraId` is still lost. `PASSIVE_NAMESPACES` contains the `w14` URI, so Plate's source-preservation lane *would* carry `w14:paraId` forward verbatim. The measurement shows `w14` at **`0/12`** — everything lost, `w14:paraId` and `w14:textId` included. The reason is that **the loss happens upstream of the lane**: the DOCX was imported through Mammoth into HTML and the `w14` attributes never survived the HTML step, so by the time the export's eligibility check runs there is no `w14`

markup left to preserve. **A passive-namespace allow-list can only preserve what still exists.** This is the sharpest single illustration of why the static reading and the measurement are not substitutes.

Disagreement 3 — `mc` at 1/4, not 0/4, for Fidus, canvas-editor and WordCraft. §3.9 and §3.10 say these editors do not handle `mc:AlternateContent` at all, which predicts 0/4. The measurement says 1/4: the **`mc:Ignorable` attribute survives** while `AlternateContent`, `Choice` and `Fallback` all vanish. The attribute survives because it is an attribute on the root element and these editors regenerate the root element from a template that happens to carry an `mc:Ignorable` list forward — while the body structure that `mc` governs is rebuilt from a model that has no `mc` concept. docs.plus and Plate.js score 0/4: Mammoth drops the attribute too.

Disagreement 5 — office-open and Docen keep the `mc` construct, and the code reading said they would not. §3.5 reported that *"no `mc:AlternateContent` branch selector was found in the DOCX layer; `mc:` content reaches the paragraph-child `default:` branch"* — which predicts the whole construct is dropped. The measurement says the opposite: **office-open and Docen keep all four `mc` names**, and the `mc:AlternateContent` element survives in their output with both branches intact.

The likely explanation is the escape hatch documented in the same section: a drawing the parser cannot model becomes an `unsupportedDrawing` node carrying the **raw XML**, which is re-emitted on save (`packages/docx/src/parts/drawing/drawing-parse.ts:225-284`; `packages/docx/src/parts/paragraph/run/unsupported-drawing-run.ts:163`). The corpus's `mc:AlternateContent` sits inside a `w:r` and contains a drawing, so it plausibly reached the model by that route rather than through the paragraph-child switch.

This is the one place in this study where the code reading overstated the loss rather than understating it, and it is worth stating in the same breath as the other four: the static reading is a map of the code, not of the data, and a construct that is preserved *as an opaque blob* is invisible to a source-level reading of the typed model.

Disagreement 4 — `w16du` is lost by LibreOffice. LibreOffice is the round-trip baseline, not one of the twelve editors, and it is in the matrix because a baseline is worth having. It keeps `w`, `a`, `dgm`, `c`, `wps`, `v`, `o`, `ds`, `ns0` and every package namespace, and it loses `w16du`, `w16cid`, `cs`, `w10` and `w15`. **Microsoft's own file format, round-tripped through the closest open implementation to Word, loses the 2023 UTC-date attribute.** That is a fair measure of how young that namespace is.

4.5 The code reading is wrong in both directions

The four disagreements above all run one way: the source reading predicts a **larger** loss than the file shows. The opposite also happens, and the matrix in §6 — rebuilt in this edition from the measurement rather than from the source reading — makes it visible for the first time.

Twenty-seven cells in the earlier matrix said *dropped* where the measurement finds the namespace kept, and **twenty** said *kept or flattened* where the measurement finds it gone. The sharpest of the first group is **Plate.js and the OMML**: Mammoth's importer reports *"An unrecognised element was ignored: m:oMath"*, Plate.js surfaces that as a `converter-message` diagnostic, and the earlier edition of this report therefore listed Plate.js among the editors that drop `m` outright — but the measurement finds **all four `m:` names** in Plate's output package. The pattern repeats across eleven groups; the earlier cell is shown in bold:

Namespace	Editors whose earlier cell was wrong	What the measurement finds
`pic`	Fidus (**D**), canvas-editor (**D**), docs.plus (**D**)	all six `pic:` names — and the image bytes too, though the media file is renamed (`image-659975.png`, `zsakc18onfp21xzadj5t0.png`, `image-ZwEGiAbAoIUtGFAOXxOfH.png`)
`wp`	Fidus (**D**), canvas-editor (**D**), docs.plus (**D**)	3/5, 5/5 and 4/5 respectively
`w15`	Fidus (**D**), canvas-editor (**D**), office-open (**D***), Docen (**D**)	all four keep `w15:restartNumberingAfterBreak` — in `word/numbering.xml`, not in the document body, which is why a body-only reading misses it
`w16cid`	WordCraft (**D**)	the attribute is present, in `word/commentsIds.xml`
`w14`	WordCraft (**D**), office-open (**D***), Docen (**K***)	WordCraft keeps all twelve names; office-open and Docen keep 10/12 — Docen's earlier cell claimed a *full* keep and was wrong in the opposite direction

Namespace	Editors whose earlier cell was wrong	What the measurement finds
`ns0`	office-open (**D**), Docen (**D**), WordCraft (**D**)	both foreign names
`rels`	Fidus (**D**), canvas-editor (**D**), docs.plus (**D**)	both names
`a`	Fidus (**D**), docs.plus (**D**)	11/60 and 42/60 — partial, but not nothing
`m`	Plate.js (**D**)	all four `m:` names — the importer ignored the element and the export lane still carried it
`w:ins` and the other revision wrappers	office-open (**D**), Docen (**D**)	all three revision markers survive
`dgm`, `c`, `v`, `o`	SuperDoc (**D+G**)	every name present in the package — SuperDoc's diagnostic belongs to the *renderer*, not to the writer

And twenty cells run the other way: the earlier reading said *kept or flattened* where the measurement finds the namespace gone. **Twelve of them are Plate.js** — `mc`, `pic`, `wp`, `v`, `o`, `r`, `w10`, `w14`, `w15`, `w16cid`, the revision wrappers and `w:sdt` — which is why Plate is the only editor in the study that loses the image outright. WordCraft loses `w10`. Fidus loses `m` (contrary to the earlier reading that it converts OMML to MathML) and `w:sdt`; canvas-editor loses `v` (contrary to the earlier reading of a partial VML model), `w14` (contrary to the earlier reading that it reads `w14:checkbox`) and `w:sdt`; docs.plus loses `mc` and `w:sdt`.

The mechanism behind both directions is the same one. A construct that reaches the model as an *opaque payload* and is re-emitted verbatim is invisible to a source-level reading of the typed model — in either direction: the reading cannot see that it survived, and it cannot see that it did not. So the honest conclusion is not "the code reading understates the loss" but the sharper, two-sided one: **a code reading is a map of the code, and a construct's survival in a package can be established only by measurement.**

That is why §6's letters are now derived from the measurement and the parentheticals carry the mechanism. It also retires a small piece of the report's furniture: Plate.js's passive `w14` namespace and SuperDoc's closed engine are no longer represented in §6 as survivals the source reading predicts, but as **mechanisms that could not be verified** — marked with a `?` suffix — while the survival itself is stated by §4.2 and checked automatically before the table is written.

---

5. The consequences, namespace by namespace

This section states what each namespace *does* in a document and what it costs when it is lost. Each subsection ends with the editors that lose it and the ones that keep it, so the claim is falsifiable in both directions.

5.1 `mc` — Markup Compatibility: which branch to believe

What it does. `mc:AlternateContent` lets a producer offer two encodings of the same thing and let the consumer pick. The `mc:Choice` elements carry a `Requires` attribute naming the namespace prefixes the consumer must understand; the consumer takes the **first `Choice` whose `Requires` it satisfies**, and falls back to `mc:Fallback` only if it satisfies none. `mc:Ignorable` on the root declares prefixes a consumer may ignore if it does not understand them.

Why it is the most consequential namespace in the corpus. The corpus's `mc:AlternateContent` sits inside a `w:r` and offers two encodings of a text box: `mc:Choice Requires="wps"` (a modern Word 2010 shape) and `mc:Fallback` (a legacy `v:shape` with a `v:textbox`). **Both branches carry visible text**, and they carry *different* text — *"Text box content"* versus *"AlternateContent fallback text"*. So:

- An editor that takes the **wrong branch** does not merely lose a structure; it **shows the reader the wrong text**.
- An editor that **flattens** the construct loses the ability to re-serialize it, but keeps whatever it chose.
- An editor that **drops** it loses the text box entirely.

What the twelve editors do — and what the thirteenth participant does.

Behaviour	Editors	Evidence
Takes the first `Choice` **without evaluating** `Requires`	WordCraft	`wordcraft/crates/docx/src/read/story.rs:161-175`
Takes the `Fallback` — **the opposite branch from Word**	Plate.js, docs.plus (both via Mammoth)	`mammoth/lib/docx/office-xml-reader.js:39-42`
Evaluates `Requires` correctly	eigenpal	`drawing-projection.ts:312`, `:549`, `:1291-1340`
Takes the `Choice` for display, splices the original bytes back	GenOffice	`parse.ts:5915-5934`; `patch.ts:1644-1648`
Preserves the structure, fails closed with a diagnostic	SuperDoc	`drawing-taxonomy.ts:255-256`, `:372-376`
Captures `mc:AlternateContent` verbatim and replays it while the node is untouched; takes the first Choice when it is edited	Folio	`alternateContentAttrs.ts:28-69`; `xmlParser.ts:571-594`
Drops the construct	Fidus, canvas-editor, WordCraft	`fidus/.../convert.js:1098`; `canvas-editor-plugin/.../importDocx.ts:1271-1281`; `wordcraft/.../read/story/runs.rs:100-107`
Drops the construct and the `mc:Ignorable` attribute	docs.plus, Plate.js	`mammoth/lib/docx/office-xml-reader.js:39-42`
Keeps the construct whole, with no branch selector of its own — it reaches the model as an opaque `unsupportedDrawing` payload and is re-emitted verbatim	office-open, Docen	`office-open/.../drawing-parse.ts:225-284`; `unsupported-drawing-run.ts:163`
Re-serialises the construct through the SDK's part model, keeping both branches	Docxodus	`Docxodus/DocxSession.cs:838-842` — the part is flushed from its `XDocument` and the package re-saved

Measured. `mc` is kept whole by SuperDoc, Folio, GenOffice, eigenpal, office-open, Docen, **Docxodus** and LibreOffice; kept at 1/4 (the `mc:Ignorable` attribute only) by WordCraft, Fidus and canvas-editor; and **dropped entirely by docs.plus and Plate.js**.

The block's fate is not merely present-or-absent — it is *how* it survives. Comparing the `mc:AlternateContent` element byte for byte against the corpus's:

Editor	The `mc:AlternateContent` block	Difference from the corpus
GenOffice	**byte-identical** (1,069 bytes)	—
SuperDoc	1,113 bytes	one added `w14:paraId` attribute
Folio	1,525 bytes	namespace declarations re-bound onto the element
eigenpal	1,055 bytes	`wp:inline` attributes re-ordered
office-open	1,133 bytes	`xmlns:a` declared onto the element
Docen	1,133 bytes	`xmlns:a` declared onto the element
Docxodus	1,080 bytes	**equivalent to the corpus after normalising self-closing-tag spacing** (11 `/>` become `/>`); both branches intact
WordCraft, Fidus, canvas-editor	**gone**	only the `mc:Ignorable` attribute survives
docs.plus, Plate.js	**gone**	the attribute goes too

Every participant that keeps the structure keeps *both* branches — the seven in the table above plus LibreOffice, eight in all.

The cost. For an editor that flattens, the cost is that the document can no longer round-trip to a different consumer — the *other* branch is gone. For an editor that takes the wrong branch, the cost is that the reader sees the wrong text. For an editor that drops it, the cost is that a text box, and the content in it, is gone.

5.2 w16du — Word 2023 UTC dates: one attribute, four uses

What it does. `w16du:dateUtc` carries a revision's timestamp in UTC, alongside the local-time `w:date` that `w:ins`, `w:del`, `w:moveFrom` and `w:moveTo` carry. It exists because a revision's local-time stamp is ambiguous across time zones; the UTC stamp is the authoritative one. Microsoft added it in 2023.

Why it is the sharpest probe. It is **one attribute**, in a namespace that is three years old, in a namespace that appears in **no** editor's dispatch table, on a wrapper element (`w:ins`, `w:moveTo`) that some editors model and some do not. It is easy to lose accidentally and it is easy to lose silently, and losing it is invisible on the page — the revision still shows, with a timestamp, just not an unambiguous one.

Who keeps it. SuperDoc, Folio, GenOffice, eigenpal — four editors — **and Docxodus**. **Folio and GenOffice keep it as a *consequence* of their architecture rather than as a decision**: Folio's `w:ins` wrapper is carried by the verbatim sink, and GenOffice splices the original bytes, so the attribute rides along inside them. eigenpal keeps it because `w:ins` is a typed element and unknown attributes on typed elements are retained. **Docxodus is the only participant in the study that keeps it by decision**: its revision layer declares a typed `XName` for the attribute (`Docxodus/Internal/RevisionOps.cs:37`) and rejects a malformed value with an `invalid_revision_date_utc` diagnostic (`RevisionOps.cs:565`).

Who loses it. office-open, Docen, WordCraft, Fidus, canvas-editor, docs.plus, Plate.js **and LibreOffice** — eight.

But the loss is not universal inside Docxodus either. The same product *drops* the attribute in its IR diff renderer, which strips *"non-w-namespace extension attributes on revision elements (e.g. Word 2023's `w16du:dateUtc`)"* when normalising a preserved original clone (`Docxodus/Ir/Diff/IrMarkupRenderer.cs:10795`). **The package keeps it; the diff renderer drops it.** That is the package-versus-render distinction in its sharpest form: one product, two layers, opposite answers on the same attribute.

The cost. Revision timestamps become ambiguous rather than lost, and the ambiguity is invisible. This is the clearest example in the study of a namespace that is *entirely* a victim of accident: none of the eight editors has a decision about `w16du`, because none of them has ever heard of it.

5.3 w14 — the identity namespace

What it does. Three separate things, all from Word 2010:

- **`w14:paraId` and `w14:textId`** — a paragraph's identity. Comments, list membership and cross-references hang off the paragraph id. It is not decoration; it is a key.
- **`w14:textOutline`, `w14:solidFill`, `w14:prstDash`, `w14:round`** — text effects (outline, fill, dashed underline).
- **`w14:ligatures`, `w14:numForm`, `w14:numSpacing`, `w14:stylisticSets`, `w14:cntxtAlts`** — OpenType typography controls, exposed by SuperDoc as addressable format attributes (`inline-run-patch.ts:388-392`).

The identity half is the load-bearing half. SuperDoc uses `w14:paraId` as the identity substrate of its whole model (`sd-nodes.ts:45-47`; `schemas.ts:1237`). Folio reads and writes it as paragraph identity, matching prefixes **by URI** (`paraIdAttribute.ts`; `wordprocessingPrefixes.ts:153`). canvas-editor-plugin reads exactly one corner of it — `w14:checkbox` on content controls (`importDocx.ts:1421`, `:1436`, `:1440-1441`).

Measured. `w14` is kept whole by SuperDoc, GenOffice, eigenpal, WordCraft **and Docxodus**; kept at 10/12 by Folio, office-open, Docen and LibreOffice (losing exactly `@paraId` and `@textId`); and **lost entirely by Fidus, canvas-editor, docs.plus and Plate.js**.

The cost. Losing `w14:paraId` does not visibly damage the page; it damages the *editing* semantics. A comment whose range is anchored to a paragraph id that no longer exists may lose its anchor. Losing the typography controls costs the reader the typographic intent of the text.

5.4 w15, w16cid, w16cex — the three smallest extensions

What they do. w15:restartNumberingAfterBreak (Word 2012) tells the numbering engine to restart a list after a section break. w16cid:durableId (Word 2016) gives a comment a durable identity across edits, so a comment survives being moved or renumbered. w16cex (Word 2018) carries comment-extension metadata, and in this corpus is **declared and mc:Ignorable-listed but never used** — a weak probe, flagged as such.

Measured.

- w15 — kept by SuperDoc, Folio, GenOffice, eigenpal, office-open, Docen, Fidus, canvas, **Docxodus**. Lost by WordCraft, docs.plus, Plate.js, LibreOffice.
- w16cid — kept by SuperDoc, Folio, GenOffice, eigenpal, WordCraft, **Docxodus**. Lost by office-open, Docen, Fidus, canvas, docs.plus, Plate.js, LibreOffice.
- w16cex — no elements, no attributes; **not scored**.

The cost. w15 lost: a list continues across a break it should restart at. w16cid lost: comment identity is re-derived from position, so a moved comment can be re-attached to the wrong text. Both are invisible until someone edits.

Note the irony of w16cid. It is kept by WordCraft — which has no w16du handling at all — and lost by office-open and Docen, which *do* list w16cid in their canonical prefix table. **A canonical-prefix table is not a model.** office-open normalises the w16cid URI and then has nowhere to put the attribute.

5.5 dgm — SmartArt: the four-part diagram

What it does. SmartArt is not one element. It is **four parts** — data1.xml (the diagram's content: points, connections, text), layout1.xml (the layout algorithm), quickStyle1.xml and colors1.xml — plus a dgm:relIds element in the document that points at all four by relationship id. The parts are meaningless separately: data1.xml is a graph, and layout1.xml says how to lay it out.

Why it is the second-sharpest probe. It is the only construct in the corpus that is *inherently* a set of coordinated parts. An editor can keep the parts and still lose the construct (if the dgm:relIds pointer is dropped), and can keep the construct and still lose the parts (if they are dropped but the pointer survives, which is a dangling reference).

Measured. dgm is kept whole by SuperDoc, Folio, GenOffice, eigenpal, office-open, Docen, WordCraft, **Docxodus** and LibreOffice, and **dropped entirely by Fidus, canvas-editor, docs.plus and Plate.js** — 35 distinct names, 69 element uses, four parts, gone.

But "kept" hides three different acts.

Editor	What "kept" actually means
GenOffice, Folio, SuperDoc	The parts are returned **byte-for-byte** — the author's bytes, untouched.
eigenpal	The parts are modelled as a summary/preview, the original bytes preserved.
office-open, Docen	The parts are carried as **opaque blobs** — raw XML the parser did not understand, re-emitted verbatim.
WordCraft	The diagram is **regenerated from WordCraft's own model** (read/diagram.rs + a SmartArt writer). This is not preservation — it is re-authoring. The result looks like the original and is not the original.
SuperDoc	Preserved in the file, **drawn as a placeholder**, with the diagnostic render.drawing.unsupported-object (drawing-taxonomy.ts:328-332).
Docxodus	The four diagram parts are **copied byte-for-byte** — they are not among the ten parts the projector rewrites — while the dgm:relIds pointer in document.xml is re-serialised. **The payload is the author's; the pointer is Docxodus's.** Its *renderer*, however, substitutes a [SMARTART] placeholder for the diagram (see *Rendering Versus Word*).

The cost. For the four editors that drop it: a diagram becomes a blank rectangle. For the four that keep it but render a placeholder (SuperDoc and Docxodus, and by extension Folio's bounded 128-shape preview and eigenpal's summary): **the file is right and the screen is wrong** — the user sees a placeholder, sends the file on, and the recipient sees the real diagram. For WordCraft: the diagram is *replaced*, and the user is not told.

5.6 c — the chart, and its two orphaned companions

What it does. The chart part holds the axes, series, categories and value caches. Two **companion parts** — `word/charts/style1.xml` and `word/charts/colors1.xml` — hold the chart's appearance, in the `cs` namespace.

The companion parts are the point. A chart without its style and colour parts is a chart that renders with the default theme instead of the author's design. Keeping `chart1.xml` while dropping `style1.xml` and `colors1.xml` produces a file that is *structurally valid* and *visually different* — and no part-level checker will notice.

Measured.

- `c` — kept whole by SuperDoc, Folio, GenOffice, eigenpal, office-open, Docen, WordCraft **and Docxodus**; **dropped entirely** by Fidus, canvas-editor, docs.plus and Plate.js; kept at 39/40 by LibreOffice (losing `c:layout`).
- `cs` — kept whole by SuperDoc, Folio, GenOffice, eigenpal, office-open, Docen **and Docxodus**; **dropped by WordCraft**, Fidus, canvas-editor, docs.plus, Plate.js **and LibreOffice**.

WordCraft is the only editor that keeps the chart and loses its companions. That is the mechanism behind its chart rendering differently from the original while the chart part itself measures as pass-through — a finding from the first report, now with its namespace-level explanation.

Docxodus keeps both, and for a structural reason worth stating. `word/charts/style1.xml` and `word/charts/colors1.xml` are not among the ten parts the projector rewrites, so they are copied byte-for-byte — the author's chart design, not a re-derived one. This is the strongest form of the companion-part result available: not only is nothing lost, the *appearance* is literally the author's bytes.

The cost. Dropping `c` and `cs` together: the chart becomes a placeholder or nothing. Dropping `cs` alone: the chart survives and looks wrong. The second is worse, because nothing reports it.

5.7 v and o — the legacy namespaces

What they do. `v` is **VML**, the drawing format Word used before DrawingML: `v:shape`, `v:imagedata`, `v:textbox`. It is the *legacy fallback* branch of the corpus's `mc:AlternateContent`, and it is still how watermarks and horizontal rules are drawn. `o` is **OLE**: `o:OLEObject`, the `<w:object>` wrapper around an embedded foreign document — the corpus's is a real embedded `.docx`, plus `o:title`.

Measured.

- `v` — kept whole by SuperDoc, Folio, GenOffice, eigenpal, office-open, Docen, **Docxodus** and LibreOffice; **2/3 by WordCraft** (losing `v:textbox`); **dropped entirely** by Fidus, canvas-editor, docs.plus and Plate.js.
- `o` — kept whole by SuperDoc, Folio, GenOffice, eigenpal, office-open, Docen, WordCraft, **Docxodus** and LibreOffice; **dropped entirely** by Fidus, canvas-editor, docs.plus and Plate.js.

The asymmetry worth noting. WordCraft keeps `o` but loses `v:textbox` — because it has a VML reader (`read/story/vml.rs`, 626 lines) that models shapes and images but not text boxes. And canvas-editor keeps *corners* of `v` — watermarks (`v:shape` with `v:textpath`/`v:imagedata`) and rules (`v:line`, flat `v:rect`, `importDocx.ts:782-820`, `:1199-1200`) — while losing the rest. **A namespace can be partially understood in a way that is worse than not understanding it at all**, because the partial model produces a plausible-looking document with a hole in it.

The cost. VML lost: the legacy rendering of every modern drawing is gone — but if `mc` was handled, the modern branch still draws. OLE lost: the embedded document is gone; the corpus's embedded `.docx` and `.xlsx` are unreachable.

5.8 `m` — OMML math

What it does. Office Math Markup Language: `m:oMath` (an inline equation), `m:oMathPara` (a display equation), `m:r` and `m:t` (the runs and their text). It is the only namespace in the corpus whose content is *mathematics*, and therefore the only one where a lossy conversion is visible as a wrong answer rather than as a missing picture.

Measured. `m` is kept whole by SuperDoc, Folio (converted to MathML, `mathToMathml.ts`), GenOffice, eigenpal, eigenpal's dedicated OMML path (`oMML-equation.ts:770`), office-open, Docen, WordCraft (dedicated reader/writer), **Plate.js**, Docxodus and LibreOffice; and **dropped entirely by Fidus, canvas-editor and docs.plus**.

Plate.js is the surprise, and it sharpens the point. Mammoth has no math reader, so it emits *"An unrecognised element was ignored: m:oMath"*. docs.plus surfaces that as an `unsupported-element ConversionWarning`; Plate surfaces it as a `converter-message` diagnostic. **Both report it. docs.plus loses the mathematics — its package has none of the four `m:` names. Plate.js does not: its DOCX export lane writes all four.** The importer ignored the element and the writer still carried it, because the subtree reached the model by a route the importer's own diagnostic never mentions. **Reporting is not preservation, and neither is ignoring**: a namespace can be reported as unrecognised, be absent from the rendered HTML, and still be sitting in the exported package. An earlier edition of this report listed Plate.js among the editors that drop `m`; that was the code reading, and §4.5 corrects it.

Fidus is the other correction. The earlier reading said Fidus converts OMML to MathML. It does not — Fidus's output has none of the four `m:` names.

Docxodus keeps the OMML in the package and substitutes a `[MATH]` chip for it on the page. The package layer and the render layer give opposite answers about the same namespace, which is the subject of this study in a single cell.

5.9 `ns0` — the foreign namespace

What it does. Nothing standard. `customXml/item1.xml` in the corpus binds `ns0` to `http://example.com/fixture` and contains two elements (`ns0:root`, `ns0:field`). It is the custom-XML datastore payload — user data attached to the document, in a namespace the document author chose.

Why it matters. This is the test of whether a boundary is **open** (unrecognised content is carried) or **closed** (unrecognised content is discarded). It appears in **no** editor's namespace table, allow-list, or dispatch switch.

Measured. `ns0` is kept whole by SuperDoc, Folio, GenOffice, eigenpal, office-open, Docen, WordCraft, **Docxodus** and LibreOffice, and **dropped entirely by Fidus, canvas-editor, docs.plus and Plate.js**.

But the mechanisms behind that split are three different ones, and only one of them is a deliberate policy:

Editor	Why `ns0` survives	Deliberate?
GenOffice	The bytes are spliced, never re-serialized (`patch.ts:1644-1648`)	**Not a decision** — a side effect of the architecture
Folio	The undeclared name falls into the verbatim sink (`containerChildren.ts:342-346`)	**Not a decision** — a side effect of totality
eigenpal	The element is demoted to a `generic` node (`ooxml-tree.ts:1252-1261`)	**Yes** — it is the documented fallback
office-open, Docen	The part is carried as an opaque blob or by prefix-agnostic parsing	**Not a decision**
WordCraft	The whole part is copied through a manifest-gated passthrough	**Not a decision**
Docxodus	`customXml/item1.xml` is not among the ten parts the projector rewrites, so it is copied byte-for-byte	**Not a decision** — a side effect of the part boundary

The cost of losing it. Custom XML is where document metadata, content-control data bindings, SharePoint properties and compliance tags live. Losing it is silent, and it can break a document that is *structurally fine* and functionally dead.

5.10 `w` and `r` — the main vocabulary and the pointers

What `w` does. Everything a word processor is for: 309 distinct element and attribute names covering paragraphs, runs, properties, tables, numbering, styles, sections, headers, footers, notes, comments, fields, content controls and revision markup.

What `r` does. It carries the **relationship ids** — `r:id`, `r:embed`, `r:dm`, `r:lo`, `r:qs`, `r:cs`. Six names, seventeen uses, and every one is a pointer from the document body into the package: the diagram's four parts (`r:dm`, `r:lo`, `r:qs`, `r:cs`), the image (`r:embed`), the hyperlink (`r:id`).

Measured. `w` is kept whole by SuperDoc, Folio, GenOffice, eigenpal **and Docxodus**; office-open loses 1 of 309, Docen 4, WordCraft 15, LibreOffice 49, and then the cliff: Fidus 98, canvas 119, docs.plus 114, Plate.js 108. `r` is kept whole by nine participants, and by **none of Fidus, canvas-editor, docs.plus or Plate.js** beyond one or two names.

The correlation is the finding. Look at the `r` row and the `dgm`, `c`, `pic` rows: they move together. That is not a coincidence of implementation; it is the nature of the format. `r:dm` is the only thing that connects `word/document.xml` to `word/diagrams/data1.xml`. **When an editor drops the drawing parts, it must also drop the relationship ids — and when it drops the relationship ids, it drops the pointers.** So `r` is the namespace that carries the *evidence* of a drop.

The cost. Losing `w` names is the cost of losing edits, formats, tables and revisions. Losing `r` names is the cost of losing the *links* — and a document with a dangling `r:embed` is a document Word will offer to repair.

5.11 `app`, `cp`, `dc`, `dcterms` — the metadata

What they do. `app` (`docProps/app.xml`) holds application, template, company, page and word counts, total editing time. `cp` (`docProps/core.xml`) holds title, creator, created/modified, last-modified-by, revision, keywords, category. `dc` holds Dublin Core title, subject, creator, description. `dcterms` holds the created/modified timestamps in ISO form.

Measured.

- `app` — kept whole by seven participants; partially by WordCraft (8/9), Fidus (3/9), canvas (1/9), LibreOffice (4/9); **gone entirely** from docs.plus and Plate.js.
- `cp` — kept whole by eight participants; 4/5 by docs.plus and Plate.js; 3/5 by canvas; **1/5 by Fidus**.
- `dc` — kept whole by every participant except Fidus (2/4) and canvas (1/4).
- `dcterms` — **kept whole by every single participant**, including the four that drop `app` and `cp` entirely.

The `dcterms` result is the sharpest. Both `dcterms:created` and `dcterms:modified` survive in every one of the thirteen outputs, while `app` and `cp` are gutted by four editors. The explanation is mundane and worth stating because it explains a general pattern: **the two `dcterms` attributes are on the root `<cp:coreProperties>` element, and every editor that regenerates that element writes a fresh one with the timestamps filled in from its own clock.** They are not preserved — they are **re-derived**. An editor that wrote today's date into `dcterms:modified` would score 2/2 on this measurement while destroying the original timestamp.

This is the clearest limitation of a name-set measurement, and it is stated here rather than buried: **the measurement proves a name is present, not that its value is correct.** For `w14:paraId`, a re-minted id is a *different* id; for `dcterms:modified`, a re-stamped date is a *different* date. Both score as "kept".

---

6. The cross-editor matrix

Legend. K every distinct name kept and carried unchanged · R every distinct name kept but re-serialised or re-modelled — a partial loss is shown as `R (n/N)` · S flattened or summarised · D dropped, no distinct name survives · G a named warning or diagnostic is emitted · `?` a *suffix* marking a cell whose **survival is measured but whose mechanism is not verifiable from source** · — not applicable.

The letters are derived from the measurement, not from the source reading. Every cell below is checked against `out/ns_map.json` — the §4.2 measurement — by `build/report_a_matrix.py` before the table is written, so this table cannot contradict §4.2. The parenthetical in each cell is the **mechanism**, established from source in §3.

The Docxodus column is derived, not guessed. It follows from the part classification in §3.12: the ten parts the projector re-writes and the twenty-five it copies byte-for-byte. `K (part)` means the namespace's payload lives in a copied part; `R (parts K)` means the namespace spans both a rewritten part and a copied one; `R (media K)` means the media is copied but the reference to it is re-serialised.

namespace	SuperDoc	Folio	GenOffice	eigenpal	office-open	Docen	WordCraft	Plate.js	Fidus	canvas	docs.plus	Docxodus
`w`	R	R	R/K	R	R (308/309)	R (305/309)	R (294/309)	S→R	S→R	S→R	S→R	R
`mc`	R+G	R	K	R	R	R	D (attr only)	D (Fallback)	D (attr only)	D (attr only)	D (Fallback)	R
`w14`	K (identity)	R (10/12)	K	K	R (10/12)*	R (10/12)*	R	D	D	D	D	R
`w15`	R?	K	K	K	R*	R	D	D	R	R	D	K
`w16cid`	R?	K	K	K	D*	D	R	D	D	D	D	K
`w16cex`	?	K	K	K	D*	D	R	K	D	D	D	—
`w16du`	R?	K	K	K	D	D	D	D	D	D	D	R (SDK 3.5.1 models it)
`a`	K	R	K	R	R	R	R (56/60)	R (33/60)	R (11/60)	R (11/60)	R (42/60)	R
`wp`	K	R	K	R	R	R	R	D	R (3/5)	R	R (4/5)	R
`pic`	K	R	K	R	R	R	R	D	R	R	R	R (media K)
`dgm`	R+G	S	K	S	R	R	R (regen)	D	D	D	D	R (parts K)
`c`	R (def. fidelity)	S	K	S	R	R	R+G	D	D	D	D	R (part K)
`cs`	R?	K (part)	K	K	R?	R?	D	D	D	D	D	K (part)
`m`	R?	R (→MathML)	K	R	R	R	R	R (importer ignores it)	D	D	D	R
`v`	R+G	R	K	R+G	R (opaque)	R (opaque)	R (2/3)	D	D	D	D	R
`o`	R+G	K (part)	K	R	R (opaque)	R (opaque)	R	D	D	D	D	R
`w10`	R?	K	K	K	K	K	D	D	D	D	D	R
`r`	K	R	K	R	R	R	R	D	R (1/6)	R (2/6)	R (2/6)	R
`w:ins`/`w:del`/`w:moveFrom`/`w:moveTo`	R?	K+G	K (raw accept/reject)	R	R	R	R	D	S (marks)	D	D	R
`w:sdt`	R?	R	K (shell)	R	R	R	R	D	D	D	D	R
`w:altChunk`	R?	R+G	K	K	R?	R?	D	D	D	D	D	R
`ds` (customXml)	R?	K (part)	K	R?	R?	R?	R?	D	D	D	D	K (part)

namespace	SuperDoc	Folio	GenOffice	eigenpal	office-open	Docen	WordCraft	Plate.js	Fidus	canvas	docs.plus	Docxodus
`ns0` (foreign)	R?	K (verbatim)	K	K (generic)	R	R	R	D	D	D	D	K
`ct` (content types)	R?	R	R	R	R	R	R	R	R?	R?	R?	K
`rels`	R?	R	R	R	R	R	R	R	R	R	R	K

* office-open's `OOXML_CANONICAL_PREFIXES` recognises the `w14`/`w15`/`w16cid`/`w16cex` **URIs** (so a re-bound prefix is normalised), but has no *model* for their elements or attributes; `w16du`'s URI is absent from that table entirely.

How to read the `?` suffix. A `?` is an absence of public evidence about the **mechanism**, not about the survival: the namespace's names were measured, and what happened to them is known, but *how* it happened is not readable in this checkout. Nearly all of them are SuperDoc, whose engine is the closed `@superdoc/docx-engine` distribution, and office-open's and Docen's `cs`, `ds` and `w:altChunk` cells, where the source reading found no code path at all. **No cell in this matrix is unknown** — only some of their mechanisms are. The `w16cex` row is the one exception: the corpus declares that namespace and never uses it (§2.1), so there is no measurement to derive a letter from and the row carries the code reading alone.

---

7. What this means

- **There is no single serialization boundary, and the shape of the boundary predicts the loss better than the editor's reputation does.** Five distinct mechanisms appear across the twelve editors, and a **sixth** appears across the thirteen participants. They sort cleanly: **splice-and-carry** (GenOffice, and Folio's verbatim sink) loses least; **typed-plus-generic** (eigenpal) is the only design that *documents* its fallback; **closed allow-lists** (Plate.js's `PASSIVE_NAMESPACES`, Mammoth's 9-entry namespace map) lose exactly what is not on the list; **prefix-literal dispatch** (office-open, Fidus, canvas-editor) loses whatever the switch did not enumerate; **model-regeneration** (Docen, WordCraft, docs.plus) loses whatever the model did not type; and **part-boundary** (Docxodus) does not dispatch on names at all — it rewrites ten parts and copies the other twenty-five byte-for-byte, so it has no vocabulary boundary to lose across.
- **The thirteenth participant wins by not having the kind of boundary the others have.** Docxodus is the only participant to keep every distinct name of all 28 namespaces, and it does it by drawing its boundary around a set of *parts* rather than a set of *names* (§3.12). The price is equally clear: a namespace it has never heard of is carried but cannot be changed, and its *renderer* is the layer that must decide what to do with what it cannot draw.
- **"Keeps every name" and "keeps every *value*" are different claims, and the difference is where the surprises live.** `dcterms:modified` survives in all thirteen outputs — but in at least four of them it is a *freshly written* timestamp. `w14:paraId` survives in eight — and in the ones that re-mint it, it is a *different* id. The measurement in §4 proves presence, not correctness, and §5.11 names the case where that distinction bites.
- **A namespace allow-list can only preserve what still exists.** Plate.js's `PASSIVE_NAMESPACES` contains `w14`; Plate still loses every `w14` name, because the loss happened in the HTML step upstream of the lane. Any evaluation of a "source preservation" feature must ask *where in the pipeline the check sits*, not only what is on the list.
- **Silence is the default, and it is the most expensive default.** Eight editors drop `w16du:dateUtc`. Two editors report namespace losses in a typed vocabulary (Plate.js, and docs.plus's `ConversionWarning` layer over the same Mammoth importer). One editor reports drawing failures by family (SuperDoc). **The remaining nine say nothing at all** — and the corpus's `w16du`, `w15` and `w16cid` losses are exactly the ones no user would ever notice, which is precisely why the silence is dangerous. Docxodus is silent at the serialization layer for the best possible reason — it loses nothing there — but its renderer names what it cannot draw, and that is the layer where the user is actually looking.
- **The `mc` namespace is where a document stops being a document.** It is the only construct in the corpus whose two encodings carry *different visible text*, and the only one where the thirteen participants produce **seven different outcomes** on the same input: two take the wrong branch, two take the right one without checking `Requires`, three drop it, one flattens it, one preserves it with a diagnostic, one preserves it in the file while splicing the original bytes back, and one re-serialises it through a schema-typed SDK. If you are choosing an editor by namespace behaviour, this is the row to look at.
- **The editors that keep the most are not the ones that tell you the most, and the editors that tell you the most are not the ones that keep the most.** Plate.js and docs.plus share an importer and differ only in reporting; Plate keeps less and reports more. WordCraft keeps almost everything and reports nothing. Docxodus keeps everything and reports nothing about serialization — but names, in the renderer, exactly the two namespaces it cannot draw.

Reproducing this

```
build/ns_map.py                 # measured namespace survival, printed matrix
build/ns_json.py                # the same measurement as JSON (out/ns_map.json)
build/ns_tables.py condensed    # report tables: who kept what, per namespace
build/ns_tables.py partial      # report tables: the exact lost names
build/ns_group_tables.py --patch "reports/Report A - Serialization by Namespace (2026).md"
build/report_a_matrix.py        # §6: rebuild the cross-editor matrix, validated
                                #         against out/ns_map.json before patching
build/part_map.py --matrix      # the part-level matrix (the companion report)
build/doc_diff.py <editor-dir>  # element-level diff of document.xml
build/make_report.sh <report.md>  # markdown -> .docx, with the two Word fixes
```

`build/make_report.sh` is the pipeline that produces this file: pandoc writes `w:dirty="true"` on the table-of-contents field, on which Word 16.115 asks whether to update the fields (the first report recorded it as a refusal; Report B §2), so `build/fix_dirty.py` rewrites it; and pandoc sizes every table for a portrait page, so `build/retune_tables.py` re-fits the tables to the landscape content width.

The per-editor code citations in §3 refer to the pinned commits listed in the first report, under `repos/`. **None of those checkouts was modified.** Every citation in this report was re-verified against the working tree before publication.

Gaps and honest limits

1. **SuperDoc's core DOCX round-trip is not verifiable from source.** The engine is the closed `@superdoc/docx-engine` distribution. Every SuperDoc cell marked `?` in §6 is an absence of public evidence, not evidence of absence. The measurements in §4.2 stand on their own.
2. **The measurement proves presence, not correctness.** §5.11's `dcterms` case is the clearest example. A separate check — comparing *values*, not names — would be needed to catch re-minted identifiers and re-stamped timestamps.
3. **`w16cex` is a weak probe.** It carries zero elements and zero attributes in the corpus; it is declared and `mc:Ignorable`-listed only. It is excluded from the scoring rather than counted as a pass.
4. **`wp14` is declared on every part that lists it.** It appears in `mc:Ignorable` and is bound with `xmlns:wp14` to `http://schemas.microsoft.com/office/word/2010/wordprocessingDrawing`. Copying the attribute preserves a declared prefix. The corpus does not use `wp14` as an element or an attribute, and the survival matrix does not count it as a loss.
5. **Plate.js's `formats.html` plugin mappings are not present in this checkout.** At commit `08b1571f2` the DOCX plugin's `formats.html` lives in a separate package. Plate's own `PASSIVE_NAMESPACES` table is used instead, as the stronger and more directly relevant allow-list — and §4.4 shows that even that table overstates what is preserved.
6. **The `mc:Choice Requires="wps"` branch choice for SuperDoc, Folio, WordCraft and GenOffice is read from their selection code, not measured on this corpus.** Folio's doc-comment states explicitly that `mc:Requires` is *not* evaluated (`xmlParser.ts:571-576`); WordCraft's known-prefix list suggests it *is* checked (`read/story.rs:164`) but the comparison was not traced.
7. **Fidus's and docs.plus's Mammoth citations come from the identical 1.13.0 build installed under Plate's tree.** docs.plus pins the same version in `bun.lock` but its own `node_modules` is not installed, so its line numbers are cited from the identical build rather than from a file in the docs.plus checkout.
8. **This report measures serialization, not rendering.** Whether a kept construct is *drawn* is a separate question, answered in *Rendering Versus Word*. For Docxodus the two answers are unusually far apart: its package keeps every namespace, and its renderer substitutes a `[SMARTART]` placeholder and a `[MATH]` chip for two of them.
9. **Docxodus was measured through its CLI, not through its in-browser block editor.** The browser host could not be used: `convertDocxToPaginatedHtml` inside the Vite dev server fails its own asset-graph attestation (*"The runtime asset graph is malformed"*) because Vite's pre-bundling breaks `import.meta.url` resolution of `export-assets.json`. The `@docxodus/export` CLI was used instead, with the machine's existing Playwright Chrome for Testing in place of a bundled Chromium (the `@playwright/browser-chromium`

download stalled and was abandoned) and with 20 static TTFs copied out of LibreOffice's bundle in place of the shipped font directory, which the product rejects because it contains a variable font. **All three substitutions are harness interventions and are disclosed here**; the rendered pages are the product's own materialisation, photographed rather than reproduced.

10. **Docxodus is an SDK, not an editor, and the comparison is not symmetric.** It has no editing model to lose edits in, no import pipeline to lose a namespace *in*, and no diagnostic vocabulary to report with. §6's Docxodus column is **derived** from the part classification in §3.12 — which parts the projector re-writes, and which it copies — and every cell in it is checked against `out/ns_map.json` by `build/report_a_matrix.py` before the table is written. Where the mechanism is a byte-for-byte copy of a part the projector does not touch, the cell says so (`K (part)`); where the namespace spans both a rewritten and a copied part, it says `R (parts K)`.